

“Collaborative Story/Screenplay Writing Platform with Version History”

CO1 Assessment Tool 2

Submitted in the partial fulfilment for the Course of

CSA1016 - Software Engineering

to the award of the degree of

**BACHELOR OF
ENGINEERING IN
Artificial Intelligence & Data Science**

Submitted by

192424179- LITHISH KUMAR. R

Under the Supervision of

Dr. K Anita Davamani



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical
Sciences Chennai-602105**

August 2025



SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical
Sciences Chennai-602105**



DECLARATION

I, **Lithish Kumar R 192424179**, of the **Department of Artificial Intelligence and Data Science**, Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled **“Collaborative Story/Screenplay Writing Platform with Version History”** is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place: Chennai

Date:

Signature of the Students

TABLE OF CONTENTS

S. No	TOPIC
1	Project Vision & Stakeholder Identification
2	Epics and User Stories
3	Acceptance Criteria (Given–When–Then)
4	Product Backlog Prioritization
5	Sprint Backlog & Task Breakdown
6	Story Point Estimation (Fibonacci / Planning Poker)
7	Sprint Goal and Expected Deliverables
8	Project Vision & Stakeholder Identification
9	Epics and User Stories

1. Project Vision & Stakeholder Identification

1.1 Project Vision

The Collaborative Story/Screenplay Writing Platform is a cloud-based creative writing environment that enables novelists, screenwriters, editors, and creative teams to co-author narrative content in real time. The platform combines a distraction-free, format-aware editor (supporting both manuscript prose and industry-standard screenplay elements) with a full version-history system — similar in spirit to source-control systems used by software engineers — so that every draft, branch, and revision of a creative work is tracked, comparable, and recoverable. The vision is to become the single trusted workspace where creative teams can write together, experiment with alternate storylines through branching, give and receive structured feedback, and export a submission-ready manuscript or screenplay without losing a single version of their creative history.

1.2 Project Objectives

- Enable real-time, multi-user collaborative writing with conflict-free merging of concurrent edits.
- Provide complete, Git-like version history — including branching, diffing, and restoring — for every project.
- Support both prose (novel/short story) and industry-standard screenplay formatting out of the box.
- Allow structured, inline feedback and review workflows between writers, editors, and reviewers.
- Provide export capability to industry-accepted formats (PDF, DOCX, EPUB, Final Draft .fdx).
- Ensure intellectual property protection through granular access control and secure storage.

1.3 Key Stakeholders

Stakeholder	Role / Interest in the Project
Writers / Screenwriters (Primary Users)	Create, edit, and co-author stories and screenplays; primary content owners.
Co-Writers / Collaborators	Invited contributors who write, suggest edits, and comment on shared projects.
Script Editors	Review drafts, suggest structural and language changes, track revisions across versions.
Producers / Directors	Review finalized screenplays, request specific formatted exports, track scene-level changes.
Literary Agents / Publishers	Access shared drafts for review and provide feedback prior to submission or publication.
Beta Readers / Reviewers	Provide read-only or comment-only feedback on selected drafts or versions.
Platform Administrator	Manages user accounts, subscription plans, storage quotas, and platform security.
Product Owner	Owns the product backlog, defines priorities, and represents stakeholder interests.
Scrum Master	Facilitates Agile ceremonies and removes impediments for the development team.
Development Team (Frontend/Backend/DevOps)	Designs, builds, tests, and deploys the platform's features.
UI/UX Designers	Design the writing, review, and version-comparison interfaces for usability.

Stakeholder	Role / Interest in the Project
QA / Test Engineers	Validate functionality, performance, and data integrity of version history and merges.

2. Epics and User Stories

The product requirements have been decomposed into seven Epics, each representing a major functional area of the platform. Every Epic is further broken down into well-structured User Stories following the standard Agile format: “As a <user>, I want <feature>, so that <benefit>.”

EPIC-01: User Accounts, Projects & Collaboration Management

***Epic Goal:** Allow writers to register, create writing projects, and invite collaborators with defined roles.*

US-01: As a new writer, I want create an account and set up my writer profile, so that I can start my own story or screenplay projects. (Estimated: 3 pts)

US-02: As a writer, I want create a new project and choose between 'Story' or 'Screenplay' format, so that the editor applies the correct structure and formatting rules. (Estimated: 5 pts)

US-03: As a project owner, I want invite collaborators by email and assign roles (Co-Writer, Editor, Reviewer), so that the right people have the right level of access to my project. (Estimated: 5 pts)

US-04: As a project owner, I want remove or change the role of a collaborator at any time, so that I retain control over who can edit or view my work. (Estimated: 3 pts)

EPIC-02: Real-Time Collaborative Story/Screenplay Editor

***Epic Goal:** Provide a rich, real-time collaborative text editor tailored to narrative and screenplay formats.*

US-05: As a writer, I want write and edit scenes in a distraction-free editor with auto-formatting for screenplay elements (Scene Heading, Action, Dialogue, Transition), so that my screenplay follows industry-standard formatting automatically. (Estimated: 8 pts)

US-06: As a co-writer, I want see my collaborators' cursors and edits live as they type, so that we can write together in real time without overwriting each other's work. (Estimated: 8 pts)

US-07: As a writer, I want organize my story into chapters, acts, or scenes with a drag-and-drop outline panel, so that I can restructure my narrative easily as it evolves. (Estimated: 5 pts)

US-08: As a writer, I want auto-save my work every few seconds, so that I never lose progress due to a crash or lost connection. (Estimated: 3 pts)

EPIC-03: Version History & Branching

***Epic Goal:** Give every project a Git-like version history so writers can track, compare, and revert changes safely.*

US-09: As a writer, I want view a chronological history of all saved versions of my project, so that I can see how my story has evolved over time. (Estimated: 5 pts)

US-10: As a writer, I want compare two versions side-by-side with changes highlighted, so that I can clearly see what was added, removed, or modified. (Estimated: 8 pts)

US-11: As a writer, I want restore a previous version of my project, so that I can undo unwanted changes without losing later work permanently. (Estimated: 5 pts)

US-12: As a co-writer, I want create a new branch to experiment with an alternate plot or scene, so that I can try creative alternatives without affecting the main draft. (Estimated: 8 pts)

US-13: As a project owner, I want merge a branch back into the main draft, resolving conflicts manually where needed, so that approved alternate versions can become part of the final story. (Estimated: 13 pts)

EPIC-04: Commenting, Review & Feedback

***Epic Goal:** Enable inline discussion and structured feedback on any part of the manuscript or screenplay.*

US-14: As a editor, I want leave inline comments on a specific line, scene, or paragraph, so that I can give precise, contextual feedback to the writer. (Estimated: 5 pts)

US-15: As a writer, I want reply to and resolve comments left by collaborators, so that I can track which feedback has been addressed. (Estimated: 3 pts)

US-16: As a reviewer, I want suggest a text edit that the writer can accept or reject (track-changes style), so that my suggestions can be reviewed without directly altering the writer's draft. (Estimated: 8 pts)

EPIC-05: Notifications & Activity Feed

***Epic Goal:** Keep collaborators informed about relevant project activity in real time.*

US-17: As a collaborator, I want receive a notification when I am mentioned in a comment, so that I can respond quickly to feedback directed at me. (Estimated: 3 pts)

US-18: As a project owner, I want see an activity feed of edits, comments, and version restores, so that I can stay updated on my project's progress at a glance. (Estimated: 5 pts)

EPIC-06: Export & Industry-Standard Formatting

***Epic Goal:** Allow finished work to be exported in formats accepted by the publishing and film industries.*

US-19: As a screenwriter, I want export my screenplay as a properly formatted PDF or Final Draft (.fdx) file, so that I can submit it to producers or industry competitions. (Estimated: 8 pts)

US-20: As a novelist, I want export my manuscript as a formatted DOCX or EPUB file, so that I can send it to agents, publishers, or e-book platforms. (Estimated: 5 pts)

EPIC-07: Access Control, Privacy & Security

***Epic Goal:** Protect intellectual property with granular permissions and secure data handling.*

US-21: As a writer, I want set my project visibility to Private, Invite-Only, or Public-Read, so that I control who can discover or read my unpublished work. (Estimated: 5 pts)

US-22: As a platform administrator, I want enforce encrypted storage and access logging for all projects, so that writers' intellectual property remains secure and auditable. (Estimated: 8 pts)

3. Acceptance Criteria (Given–When–Then)

Clear, testable acceptance criteria have been written for the highest-priority user stories selected for the first sprint (and key stories from later sprints), using the Given–When–Then format to ensure each requirement is unambiguous and verifiable by QA.

US-02: Create a new project with format selection

Given a logged-in writer on the dashboard,

When the writer clicks 'New Project' and selects 'Screenplay' as the format,

Then the system creates a project pre-loaded with screenplay element styles (Scene Heading, Action, Character, Dialogue, Parenthetical, Transition).

Given the writer selects 'Story' as the format,

When the project is created,

Then the system loads a manuscript-style editor with chapter and scene structuring instead of screenplay elements.

US-05: Auto-formatting screenplay elements

Given a writer is editing a screenplay project,

When the writer types a line and presses Tab or Enter to change element type,

Then the editor applies correct indentation, capitalization, and spacing for the selected element (e.g., ALL CAPS and centered for Scene Heading).

Given a writer types a character name in a Dialogue block,

When the same name was used earlier in the script,

Then the editor auto-suggests the existing character name for consistency.

US-06: Real-time collaborative editing

Given two or more collaborators have the same project open,

When one collaborator types or deletes text,

Then the change is reflected on all other collaborators' screens within 2 seconds, with each user's cursor labeled by name/color.

Given two collaborators edit the same line simultaneously,

When both changes are saved,

Then the system merges non-conflicting edits automatically and flags conflicting edits for manual resolution.

US-09: View version history

Given a writer opens the 'Version History' panel of a project,

When the panel loads,

Then the writer sees a reverse-chronological list of saved versions, each showing timestamp, author, and a short auto-generated summary of changes.

Given more than 50 versions exist,

When the writer scrolls the list,

Then older versions load via pagination without noticeable delay.

US-11: Restore a previous version

Given a writer is viewing a past version in Version History,

When the writer clicks 'Restore this version',

Then the system prompts for confirmation, then creates a new version identical to the selected one, preserving all prior versions untouched.

Given restoration is confirmed,

When the restore completes,

Then all active collaborators receive a notification that the document was restored to a prior version.

US-12: Create a branch for alternate plot/scene

Given a writer is viewing the main draft,

When the writer selects 'New Branch' and provides a branch name,

Then the system creates an independent copy of the current draft that can be edited without affecting the main branch.

Given a branch exists,

When the writer switches between branches,

Then the editor loads the correct branch content and clearly labels the active branch name.

US-14: Inline commenting

Given an editor selects a portion of text in the manuscript,

When the editor clicks 'Add Comment' and submits feedback,

Then the comment is anchored to that exact text range and visible to all collaborators with comment-view permission.

Given the anchored text is later edited,

When the edit does not delete the anchored range,

Then the comment remains correctly attached to the (possibly shifted) text.

US-19: Export screenplay as PDF / Final Draft

Given a writer opens the 'Export' menu on a completed screenplay,

When the writer selects 'PDF (Industry Standard)',

Then the system generates a PDF using standard screenplay margins, Courier 12pt font, and correct page/scene numbering.

Given the writer selects 'Final Draft (.fdx)',

When the export completes,

Then the downloaded .fdx file opens correctly in Final Draft software with all scene elements preserved.

4. Product Backlog Prioritization

The full product backlog was prioritized using a combination of MoSCoW categorization and a 1–10 Business Value score, taking into account customer needs, technical dependencies, and the logical build order of the platform (e.g., authentication and project creation must exist before real-time editing or version history are usable).

ID	Epic	User Story (short)	Value (1-10)	MoSCoW	Dependency
US-02	EPIC-01	create a new project and choose between 'Stor...	10	Must	US-01
US-05	EPIC-02	write and edit scenes in a distraction-free e...	10	Must	US-02
US-01	EPIC-01	create an account and set up my writer profil...	9	Must	None
US-06	EPIC-02	see my collaborators' cursors and edits live ...	9	Must	US-05
US-03	EPIC-01	invite collaborators by email and assign role...	8	Must	US-02
US-08	EPIC-02	auto-save my work every few seconds	8	Must	US-05
US-09	EPIC-03	view a chronological history of all saved ver...	8	Must	US-08
US-11	EPIC-03	restore a previous version of my project	7	Should	US-09
US-14	EPIC-04	leave inline comments on a specific line, sce...	7	Should	US-06
US-19	EPIC-06	export my screenplay as a properly formatted ...	7	Should	US-05
US-07	EPIC-02	organize my story into chapters, acts, or sce...	6	Should	US-05
US-10	EPIC-03	compare two versions side-by-side with change...	6	Should	US-09
US-21	EPIC-07	set my project visibility to Private, Invite-...	6	Should	US-02
US-22	EPIC-07	enforce encrypted storage and access logging ...	6	Should	US-21
US-04	EPIC-01	remove or change the role of a collaborator a...	5	Should	US-03
US-12	EPIC-03	create a new branch to experiment with an alt...	5	Could	US-09
US-15	EPIC-04	reply to and resolve comments left by collabo...	5	Should	US-14
US-16	EPIC-04	suggest a text edit that the writer can accep...	5	Could	US-14
US-13	EPIC-03	merge a branch back into the main draft, reso...	4	Could	US-12
US-17	EPIC-05	receive a notification when I am mentioned in...	4	Could	US-14
US-20	EPIC-06	export my manuscript as a formatted DOCX or E...	4	Could	US-05
US-18	EPIC-05	see an activity feed of edits, comments, and ...	3	Could	US-17

5. Sprint Backlog & Task Breakdown

Sprint 1 (2-week sprint) focuses on establishing the platform's foundation: account creation, project setup, the core collaborative editor, auto-save, initial version history, and privacy controls. These stories were selected because they are all rated 'Must' or high-value 'Should' items and unblock nearly every other Epic in subsequent sprints.

5.1 Selected Sprint 1 Stories

ID	User Story	Story Points
US-02	As a writer, I want create a new project and choose between 'Story' or 'Screenplay' format...	5
US-05	As a writer, I want write and edit scenes in a distraction-free editor with auto-formatting for screenplay elements (Scene Heading, Action, Dialogue, Transition)...	8
US-01	As a new writer, I want create an account and set up my writer profile...	3
US-06	As a co-writer, I want see my collaborators' cursors and edits live as they type...	8
US-03	As a project owner, I want invite collaborators by email and assign roles (Co-Writer, Editor, Reviewer)...	5
US-08	As a writer, I want auto-save my work every few seconds...	3
US-09	As a writer, I want view a chronological history of all saved versions of my project...	5
US-21	As a writer, I want set my project visibility to Private, Invite-Only, or Public-Read...	5

5.2 Task Breakdown per Story

US-01: As a new writer, I want create an account and set up my writer profile

- Design registration & login UI (Frontend)
- Implement email/password + OAuth (Google) sign-up API (Backend)
- Build writer profile page (bio, avatar, genre preferences)
- Write unit & integration tests for auth flow

US-02: As a writer, I want create a new project and choose between 'Story' or 'Screenplay' format

- Design 'New Project' modal with format selector
- Define data schema for Story vs Screenplay projects
- Implement backend endpoint to initialize project with correct template
- Load correct editor template (manuscript vs screenplay) on project open

US-03: As a project owner, I want invite collaborators by email and assign roles (Co-Writer, Editor, Reviewer)

- Design collaborator invite modal with role dropdown
- Implement email invitation service (send + accept link)
- Implement role-based access control (RBAC) middleware
- Test invite, accept, and role-enforcement scenarios

US-05: As a writer, I want write and edit scenes in a distraction-free editor with auto-formatting for screenplay elements (Scene Heading, Action, Dialogue, Transition)

- Build rich-text editor core using a CRDT-based framework
- Implement screenplay element styles (Scene Heading, Action, Dialogue, etc.)
- Implement Tab/Enter element-switching logic and keyboard shortcuts
- Cross-browser formatting QA

US-06: As a co-writer, I want see my collaborators' cursors and edits live as they type

- Integrate CRDT/WebSocket sync layer for real-time edits
- Implement live cursor presence indicators with user color/name labels
- Implement automatic conflict-free merge for concurrent edits
- Load/stress test with 10 simulated concurrent editors

US-08: As a writer, I want auto-save my work every few seconds

- Implement debounce-based auto-save every 5 seconds
- Implement offline-edit local caching with sync-on-reconnect
- Add visual save-status indicator (Saving.../Saved)
- Test auto-save under network interruption

US-09: As a writer, I want view a chronological history of all saved versions of my project

- Design Version History side panel UI
- Implement backend versioning service (snapshot on save)
- Generate auto-summaries of changes per version (diff-based)
- Implement pagination for long version lists

US-21: As a writer, I want set my project visibility to Private, Invite-Only, or Public-Read

- Design project visibility settings UI (Private/Invite-Only/Public-Read)
- Implement backend enforcement of visibility rules on all read endpoints
- Add audit logging for visibility changes
- Security test to confirm private projects are inaccessible to non-invitees

6. Story Point Estimation Using Story Points

Story points were assigned using Planning Poker with the Fibonacci sequence (1, 2, 3, 5, 8, 13, 21), where the team estimated relative effort, complexity, and uncertainty rather than absolute time. The whole team (Product Owner, Scrum Master, Developers, QA, and UX Designer) independently proposed estimates; divergent votes were discussed and re-voted until consensus was reached, following standard Planning Poker practice.

6.1 Estimation Scale Reference

Points	Relative Complexity Meaning
1	Trivial change, no unknowns (e.g., copy/label update).
2	Small, well-understood task with minimal integration.
3	Small feature with minor UI/backend work and clear requirements.
5	Medium feature involving both frontend and backend, some design decisions.
8	Complex feature with real-time sync, multiple edge cases, or new architecture.
13	Very complex feature with significant unknowns, cross-cutting impact, or algorithmic difficulty.

6.2 Story Point Justification (Sprint 1 Stories)

US-01: 3 pts — standard auth flow using well-known libraries; low uncertainty, minimal integration risk.

US-02: 5 pts — requires distinct data models and template loading for two project types; moderate design decisions.

US-03: 5 pts — needs email invitation service plus role-based access control; moderate backend complexity.

US-05: 8 pts — core editor with screenplay-specific auto-formatting logic; high complexity and many edge cases.

US-06: 8 pts — real-time multi-user sync (CRDT/WebSockets) with live cursors; significant technical complexity and testing effort.

US-08: 3 pts — builds directly on the existing editor; well-understood debounce/save pattern.

US-09: 5 pts — requires a new versioning/snapshot service and diff-summary generation; moderate new-architecture effort.

US-21: 5 pts — visibility rules must be enforced across every read endpoint; moderate security-testing overhead.

Total Sprint 1 Capacity Committed: 42 story points

This total falls within the team's historical velocity range of 38–42 points per two-week sprint (assumed for a 5-person cross-functional team), leaving a small buffer for code review, bug fixes, and sprint ceremonies.

7. Sprint Goal and Expected Deliverables

7.1 Sprint Goal

“Deliver a working foundation of the Collaborative Writing Platform where a writer can register, create a Story or Screenplay project, invite a collaborator, and co-write in real time with auto-save and a first version of Version History — enabling an end-to-end demo of collaborative writing with recoverable drafts.”

7.2 Expected Sprint Deliverables

- Fully functional user registration, login, and writer profile module.
- Project creation flow supporting both 'Story' and 'Screenplay' formats with correct templates.
- Collaborator invitation system with role-based permissions (Co-Writer, Editor, Reviewer).
- Real-time collaborative editor with live multi-user cursors and automatic conflict-free merging.
- Auto-save functionality with visible save-status indicator and offline resilience.
- Initial Version History panel showing chronological, timestamped project snapshots.
- Project visibility/privacy controls (Private, Invite-Only, Public-Read) enforced on the backend.
- A working end-to-end demo: two users co-writing the opening scene of a sample screenplay live.

7.3 Definition of Done

- Code is peer-reviewed, merged to the main branch, and passes CI checks.
- Unit and integration tests written and passing for each story (minimum 80% coverage on new code).
- Acceptance criteria for each story verified by QA using the Given–When–Then scenarios in Section 3.
- Feature deployed to the staging environment and demonstrated at the Sprint Review.
- No critical or high-severity bugs open against the completed stories.

7.4 Sprint Timeline Overview

Day	Activity
Day 1	Sprint Planning — finalize sprint backlog and task assignments (this document).
Day 2–4	Auth, project creation, and editor foundation development.
Day 5–7	Real-time collaboration (CRDT/WebSocket sync) and auto-save implementation.
Day 8–9	Version History service and privacy/visibility controls.
Day 10	Daily stand-ups throughout; mid-sprint backlog refinement.
Day 11–12	Integration testing, bug fixing, and QA verification against acceptance criteria.
Day 13	Sprint Review — demo to stakeholders.
Day 14	Sprint Retrospective — process improvement discussion.